

RAG AGAINST THE MACHINE — 42 PROJECT SERIES

VOLUME 1 OF 3

Theory & Foundations

Language models, retrieval mathematics, chunking, and evaluation —
everything to understand before writing code

Covers: LLMs and tokenization • hallucination and the knowledge problem • the four stages of RAG
• inverted indexes • TF-IDF and BM25 derived with worked examples • analyzers for code corpora •
chunking theory • embeddings and hybrid retrieval • recall@k and IoU • grounded generation with
Qwen3-0.6B • the vLLM corpus

Companion study guide — subject version 2.0 — not an official 42 document

Contents

How to Use This Guide	6
Why theory first?	6
The project at a glance	7
The four-stage mental model	7
Language Models from First Principles	9
Tokens, not words	9
The transformer in one page	9
Autoregressive generation and sampling	10
The KV cache — why context costs memory	10
Chat templates: how a “conversation” becomes a sequence	10
Thinking mode in Qwen3	11
The Knowledge Problem: Why LLMs Hallucinate	12
Parametric knowledge and its limits	12
Hallucination is not a bug, it is the objective	12
The three classical remedies	12
Grounding: shifting the burden of proof	13
RAG: Anatomy of the Architecture	14
Stage 0 — the corpus	14
Stage 1 — indexing	14
Stage 2 — retrieval	15
Stage 3 — augmentation	15
Stage 4 — generation	15
The data contracts between stages	16
Information Retrieval Fundamentals	17
The bag-of-words model	17
The vocabulary mismatch problem	17
The inverted index	18
Zipf’s law, stopwords, and why frequency must be discounted	18
Precision, recall, and ranked retrieval	18
TF-IDF from Scratch	19
The two intuitions	19
A worked example, by hand	19

TF-IDF as a vector space	20
Where TF-IDF falls short	20
BM25: The Industry Workhorse	21
The formula	21
Fix #1 — term-frequency saturation (k_1)	21
Fix #2 — pivoted length normalization (b)	22
The smoothed IDF	22
Worked example	22
Implementing vs importing	23
Tokenization and Normalization for Search	24
The golden rule: one analyzer, two call sites	24
Stage 1 — token extraction	24
Stage 2 — identifier splitting	24
Stage 3 — case folding and stopwords	25
Stage 4 — stemming (optional, measurable)	25
What the analyzer must NOT do	25
Chunking Theory	27
Why chunk at all?	27
The size trade-off	27
Overlap	27
Strategy A — Markdown and prose	28
Strategy B — Python code	28
Invariants your chunker must guarantee	29
Dense Retrieval and Embeddings	30
How embedding models are trained	30
Cosine similarity and normalization	30
Lexical vs dense: complementary failure modes	30
Fusing rankings: Reciprocal Rank Fusion	31
Evaluation: Recall@k, IoU, and Friends	32
The matching predicate: same file + IoU ≥ 0.05	32
Recall@k, exactly	32
Why recall (and not precision) is the graded metric	33
The metrics you will not be graded on (but should know)	33
Grounded Generation with a 0.6B Model	34
Know your instrument	34

Prompt architecture for grounded QA	34
Context budgeting	35
Decoding and post-processing	35
Hallucination under RAG: the failure taxonomy	35
Knowing the Corpus: vLLM	36
The map of the repository	36
What the corpus's shape implies for your system	37
One Question, End to End: A Worked Trace	38
Step 0 — what exists before the question arrives	38
Step 1 — the query through the analyzer	38
Step 2 — BM25 scoring	39
Step 3 — the grader's view	39
Step 4 — augmentation and the prompt	40
Step 5 — what failure would have looked like	40
Defense Drill: Forty Questions You Should Be Able to Answer	42
Language models and RAG (Chapters 2-4)	42
Retrieval theory (Chapters 5-7)	42
Chunking, analysis, evaluation (Chapters 8-11)	43
Generation and system design (Chapters 12-13)	43
The Subject, Decoded	45
The five decisions that determine your score	45
Glossary and Further Reading	47
Glossary	47
Further reading	48
Appendix: The Mathematical Toolbox	49
Logarithms, and why IR is full of them	49
Ratios of counts as probabilities	49
Saturating functions	49
Vectors, dot products, cosine	50
Intervals: intersection, union, IoU	50
Averages that lie, averages that don't	50
Appendix: Notation, Symbols, and a Study Plan	51
Symbols used throughout this volume	51
A one-week study plan	51

How the three volumes cross-reference	51
---	----

CHAPTER 1

How to Use This Guide

Read the whole document before writing any code: each stage feeds the next. — the subject, and it means it.

This is Volume 1 of a three-volume companion to the 42 project “**RAG against the machine**” (subject version 2.0). The project asks you to build a complete Retrieval-Augmented Generation system: ingest and index the vLLM 0.10.1 repository, retrieve the most relevant snippets for a natural-language question, hand those snippets to the small local model **Qwen/Qwen3-0.6B**, and measure retrieval quality with `recall@k`. The moulinette — the automated grader — demands at least **80% recall@5 on documentation questions** and **50% recall@5 on code questions**, with indexing under 5 minutes and 200 questions searched in under 90 seconds.

The three volumes divide the work the way you should divide your own effort:

- **Volume 1 — Theory & Foundations** (this book): everything you need to *understand* before you type a line of code. Language models, the knowledge problem, the four stages of RAG, information-retrieval mathematics (TF-IDF and BM25 derived from first principles with worked numbers), tokenization, chunking theory, embeddings, evaluation metrics, and grounded generation.
- **Volume 2 — Implementation Guide**: a step-by-step build of the whole system — project bootstrap with uv, pydantic data models, the two chunkers, the BM25 index, the Fire CLI, answer generation with Qwen3-0.6B, and the end-to-end pipeline, with complete, typed, flake8/mypy-clean code.
- **Volume 3 — Evaluation, Optimization, Bonuses & Defense**: the moulinette in depth, recall debugging, performance engineering, all five bonuses implemented, the README requirements, and preparation for the peer-evaluation, including likely questions and recode scenarios.

Why theory first?

It is tempting to jump straight into code. Resist it. This project is unusual among 42 projects in that the hard part is not making the program run — a naive RAG pipeline is perhaps 300 lines of Python — but making it *score*. The pass thresholds (80% docs / 50% code `recall@5`) are calibrated so that a thoughtless implementation fails. The difference between 55% and 85% recall is almost never “more code”; it is a better chunking decision, a better tokenizer for code identifiers, or a correctly tuned BM25. Those are theory decisions. Every hour spent understanding *why* BM25 saturates term frequency, or *why* a Markdown heading is a natural chunk boundary, repays itself during tuning.

There is a second reason: the **defense**. During peer-evaluation you must explain your system, justify every choice, and possibly modify it live (the “recode” step). The subject’s AI-instructions chapter is explicit: using generated code you cannot explain is a failing offense. This guide is written

so that after reading it you can defend every formula, every parameter, and every line of the reference implementation in your own words.

The project at a glance

Constraint	Value	Where defined
Language	Python 3.10+	Common Instructions
Package manager	uv (reviewer runs only uv sync)	Additional Requirements
Data models	pydantic	Additional Requirements
CLI framework	Python Fire	Additional Requirements
Progress bars	tqdm	Additional Requirements
Generation model	Qwen/Qwen3-0.6B (mandatory default)	Additional Requirements
Lint	flake8 + mypy (specified flags)	Makefile rules
Chunk size	--max_chunk_size, default 2000 chars, hard ceiling	VI.1
Retrieval method	TF-IDF or BM25 (at least one)	VI.1
Chunking strategies	two: Python code + Markdown/text	VI.1
Indexing time	at most 5 minutes	VII.1.2
Retrieval throughput	200 questions in at most 90 s	VII.1.2
Recall@5 docs	at least 80%	VII.1.2
Recall@5 code	at least 50%	VII.1.2
Overlap criterion	same file + IoU at least 0.05	VII.1.1

Keep this table within reach. Every design decision in Volumes 2 and 3 traces back to one of these rows, and during the defense the evaluator will probe whether you know them cold.

The four-stage mental model

The subject names four stages, and the whole series is organized around them:

1. **Indexing** — organise the data so it can be searched. You read every useful file of the vLLM repository, split each into chunks with a strategy adapted to its type, and persist a data structure under *data/processed/* that can answer queries in milliseconds.
2. **Retrieving** — match a question against the index and pull the top-k most relevant snippets, each identified by an exact file path and a character range no wider than 2000 characters.
3. **Augmenting** — filter and order those snippets and place them into the model's context window, within its token budget.

4. **Generating** — let Qwen3-0.6B read that context and produce a grounded, coherent, on-point answer, serialized as JSON conforming to the provided pydantic models.

Notice the asymmetry: stages 1–2 are *graded numerically* by the moulinette (recall@k), while stages 3–4 are judged qualitatively (coherent, grounded, on point) with explicit tolerance for the small model's reasoning limits. This tells you exactly where your engineering effort belongs: retrieval first, generation second. A perfect prompt cannot rescue a retriever that fetched the wrong file; a mediocre prompt on top of perfect retrieval usually still passes.

Note — Throughout this series, *corpus* means the vLLM repository shipped under *data/raw/vllm-0.10.1/*; *document* means one file of it; *chunk* means one contiguous character span of one file; and *source* means the triple (file_path, first_character_index, last_character_index) your system reports.

CHAPTER 2

Language Models from First Principles

Before you can see why retrieval is needed, you must see precisely what a language model is — and is not.

A **language model** (LM) is a probability distribution over sequences of tokens. Given a prefix of tokens $t_1 \dots t_{n-1}$ (written here as $t_1 \dots t_{n-1}$), the model assigns to every token v in its vocabulary a probability $P(t_n = v \mid t_1 \dots t_{n-1})$. Text generation is nothing more than repeatedly sampling from this distribution and appending the sampled token to the prefix. Everything an LLM does — answering questions, writing code, apologizing — is this one operation in a loop.

Tokens, not words

Models do not read characters or words; they read **tokens** produced by a subword tokenizer, usually Byte-Pair Encoding (BPE) or a close relative. BPE starts from bytes and greedily merges the most frequent adjacent pairs seen in training data, producing a vocabulary (about 151,000 entries for Qwen3) in which frequent words are single tokens and rare words split into pieces. The word “retrieval” may be one token; the identifier “gpu_memory_utilization” splits into several. Two consequences matter directly for this project:

- **Token budgets are real budgets.** Qwen3-0.6B has a native context window of 32,768 tokens. Your retrieved chunks, the prompt scaffolding, the question, and the generated answer must all fit inside it. As a rule of thumb for English text and code, one token is roughly 3–4 characters, so ten 2000-character chunks are roughly 5,000–7,000 tokens — comfortably inside the window, but not free: small models get *worse*, not better, as you stuff more marginal context in.
- **The model's tokenizer is not your retrieval tokenizer.** The BPE tokenizer exists to serve the neural network. For search you will write your own analyzer (Chapter 8) that splits snake_case and camelCase, lowercases, and drops punctuation — operations BPE does not do.

The transformer in one page

Modern LMs are **decoder-only transformers**. A transformer layer does two things. First, **self-attention**: every token position computes a query vector, compares it against key vectors of all previous positions, and takes a weighted average of their value vectors — the weights being softmaxed dot products. This is how token 4,000 can directly consult token 12 without information having to crawl through every position in between. Second, a position-wise **feed-forward network** (an MLP applied to each position independently) transforms the attended representation. Stack a few dozen such layers, put a token-embedding matrix at the bottom and a softmax over the vocabulary at the top, and you have GPT, Llama, or Qwen.

Qwen3-0.6B specifically: roughly 0.6 billion parameters, 28 transformer layers, hidden size 1024, grouped-query attention (16 query heads, 8 key/value heads), and a 151,936-token vocabulary. For scale, frontier models are three to four orders of magnitude larger. This is why the subject warns that the model “has known reasoning limits” and why grading “prioritizes retrieval quality, grounding and prompt strategy over perfect final phrasing”. You are working with a small brain and a big library; the project grades the library.

Autoregressive generation and sampling

Generation proceeds token by token. At each step the model outputs a vector of **logits**, one per vocabulary entry; softmax turns logits into probabilities; a **decoding strategy** picks the next token:

- **Greedy decoding** always takes the argmax. Deterministic, but prone to repetitive loops on small models.
- **Temperature sampling** divides logits by a temperature T before softmax. $T < 1$ sharpens the distribution toward the argmax; $T > 1$ flattens it. $T \rightarrow 0$ recovers greedy.
- **Top-p (nucleus) sampling** samples only from the smallest set of tokens whose cumulative probability exceeds p (e.g. 0.95), cutting off the long tail of nonsense tokens.
- **Top-k sampling** keeps only the k most probable tokens before sampling.

The Qwen team publishes recommended settings: for “thinking” mode, temperature 0.6, top-p 0.95, top-k 20; for non-thinking mode, temperature 0.7, top-p 0.8, top-k 20 — and explicitly advises *against* greedy decoding for thinking mode, which can produce endless repetition. For a grading pipeline you mostly want answers that are stable and grounded; Volume 2 discusses the trade-off and shows both configurations.

The KV cache — why context costs memory

During generation the model would naively recompute attention keys and values for the whole prefix at every step. Implementations instead cache them — the **KV cache** — so each new token costs one forward pass over one position plus attention reads over the cache. The cache grows linearly with context length and is the main memory consumer at inference time. This is worth knowing for two reasons: it explains why stuffing the context with 50 chunks is slow even when it fits, and — pleasantly — *the corpus you are indexing, vLLM, is precisely a system built around clever KV-cache management* (PagedAttention). Questions about the KV cache are likely to appear in the datasets themselves.

Chat templates: how a “conversation” becomes a sequence

A chat model is still just a next-token predictor; the conversation structure is encoded with special tokens. Qwen3 uses the ChatML convention: each message is wrapped as

ChatML wire format (what `apply_chat_template` actually produces)

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
How does vLLM schedule requests?<|im_end|>
<|im_start|>assistant
```

The model was trained so that text following *assistant* is an answer consistent with the preceding turns. When you call `tokenizer.apply_chat_template(messages, add_generation_prompt=True)` in Volume 2, this string is what is really built. Understanding it demystifies prompting: a “system prompt” is not a magic configuration channel, it is merely text placed early in the sequence, and it competes for the model's limited attention with everything else you put in the window.

Thinking mode in Qwen3

Qwen3 models are hybrids: they can emit an internal reasoning trace wrapped in `<think> ... </think>` tags before the visible answer. The tokenizer's chat template accepts a flag, *enable_thinking*. With thinking enabled the 0.6B model reasons a little better but generates far more tokens (slower, and the trace must be stripped from your JSON answer); with it disabled the model answers directly. Volume 2 measures both; the reference implementation disables thinking for dataset-scale answer generation to respect time limits, and strips any residual trace defensively. What matters here is that you can *explain* the mechanism: it is not a different model, just a template-controlled generation style the model was trained to follow.

Watch out — If you leave thinking mode on and naively dump model output into your answer field, your answers will begin with pages of deliberation. Rubric criterion “coherent and on point” dies here. Always post-process.

CHAPTER 3

The Knowledge Problem: Why LLMs Hallucinate

A language model is frozen in time. Everything it “knows” was fixed the day its training ended. — the subject’s Foreword

The subject’s foreword compresses a deep issue into two sentences. This chapter unpacks it, because “why not just ask the model directly?” is the first question any evaluator will ask you, and “because RAG” is not an answer.

Parametric knowledge and its limits

Whatever a model knows is stored **parametrically**: smeared across billions of weights as statistical associations. Parametric knowledge has three structural problems:

1. **Staleness.** Training data has a cutoff. vLLM 0.10.1 — the exact version you index — was released after (or too close to) the training cutoff of most models, and certainly the details of its internal APIs, config flags, and file layout are not reliably memorized anywhere. Retraining to refresh knowledge costs millions of dollars; re-indexing a repository costs minutes.
2. **Privacy and locality.** A private codebase was never in any training set. Your company’s incident reports, your school’s intranet — no amount of scale fixes absence.
3. **Reliability.** Even for data the model did see, recall is lossy. The model learned to produce *plausible* continuations, not *true* ones. Where memory is thin it interpolates — fluently, confidently, and sometimes wrongly. This is **hallucination**.

Hallucination is not a bug, it is the objective

It is critical to understand that hallucination is not a malfunction. The training objective — maximize the likelihood of the next token — rewards producing text that *looks like* text from the training distribution. If the training distribution contains millions of confident technical explanations and near-zero examples of “I don’t know the internals of that specific library version”, the maximum-likelihood continuation of a technical question is a confident technical explanation, true or not. A small model like Qwen3-0.6B, with less capacity to store facts, hallucinates *more*, which is precisely why the project pairs it with retrieval: the design bet of RAG is that **reading is easier than remembering**. Verifying and rephrasing evidence placed in the context window is a much easier task than recalling that evidence from weights.

The three classical remedies

Approach	Mechanism	Cost	When it wins
Fine-tuning / continued pre-training	Push new documents into the weights by further gradient descent	GPU-hours per update; risk of catastrophic forgetting; still lossy	Stable domain style/format; teaching skills, not facts
Long-context stuffing	Concatenate the entire corpus into the prompt	Quadratic-ish attention cost; 'lost in the middle' degradation; corpus must fit (vLLM repo is tens of MB — it does not)	Tiny corpora, one-off questions
Retrieval-Augmented Generation	Search an external index at question time; feed only the top-k snippets to the model	Build and maintain an index; retrieval quality becomes the bottleneck	Large, changing, or private corpora — this project

The comparison explains the project's shape. The vLLM repository is on the order of 10^4 files and 10^7 characters — hopeless to stuff into a 32k-token window, pointless to fine-tune a 0.6B model on. Indexing it once (under five minutes, per the subject) and retrieving per question is the only approach that fits the constraints. When the evaluator asks “why RAG and not fine-tuning?”, this table is your answer.

Grounding: shifting the burden of proof

RAG changes the epistemic contract. Without retrieval, an answer's authority is “the model says so”. With retrieval, every claim in the answer should be traceable to a retrieved span — the answer is **grounded**. The subject's quality bar for answers is exactly this: *coherent, grounded in the retrieved sources with no major hallucination, on point*. Grounding also gives you citations for free: your output format already carries the file paths and character ranges the answer relied on, which is precisely what MinimalAnswer serializes.

Note — Vocabulary for the defense: **parametric knowledge** (in the weights) vs **non-parametric knowledge** (in an external store consulted at inference time). RAG is a non-parametric memory bolted onto a parametric reasoner.

CHAPTER 4

RAG: Anatomy of the Architecture

Instead of stuffing more knowledge into the model, we let it reach out for the right information at the moment it answers.

The term *Retrieval-Augmented Generation* comes from a 2020 paper by Lewis et al. (Facebook AI Research), which jointly trained a dense retriever and a generator. The modern usage is broader: any pipeline that (1) retrieves evidence from an external corpus and (2) conditions a generator on it. Your project is a clean, canonical instance of the modern form, with a lexical retriever and a frozen generator. This chapter walks the full pipeline end to end, naming every component and every design decision you will face.

Stage 0 — the corpus

Everything begins with what you choose to index. The subject says: “Read the files you judge useful from the vLLM repository.” That sentence hides a real decision. The repository contains Python sources, Markdown documentation, C++/CUDA kernels, YAML configs, JSON fixtures, images, licenses. Your obligations are only two chunking strategies — Python and Markdown/text — and your recall targets are defined over *docs questions* and *code questions*. Indexing binary files is useless; indexing enormous generated files wastes your 5-minute budget; but excluding a file type entirely is a gamble that no ground-truth source lives there. Volume 2 chooses: index .py with the code chunker; .md with the Markdown chunker; and treat other small text files (.txt, .rst, .yaml, .toml, .sh, .cfg, .json under a size cap) with the text chunker, because the cost is small and the downside of a hole in coverage is a hard recall loss.

Stage 1 — indexing

Indexing transforms raw files into a searchable structure. Concretely it is a pipeline of four sub-steps:

1. **Discovery:** walk data/raw/, filter by extension and size, skip junk directories (.git, __pycache__, build artifacts).
2. **Chunking:** split each file into spans of at most max_chunk_size (default 2000) characters, recording for each chunk its exact (first_character_index, last_character_index) in the original file. Chapter 9 is devoted to this.
3. **Analysis:** convert each chunk’s text into a bag of normalized terms (Chapter 8) — lowercase, split identifiers, drop stopwords.
4. **Statistics & persistence:** compute document frequencies, term frequencies, chunk lengths; store everything under data/processed/ in a format that loads in milliseconds (the reference uses a single compressed pickle/JSON of postings; Volume 2 discusses formats).

The index is the only artifact retrieval touches. If information is destroyed here — a file skipped, an identifier not split, a chunk boundary through the middle of the one relevant paragraph — no later stage can recover it. The subject hints at exactly this: “What you keep at indexing time decides which of the two [paraphrase or verbatim identifier] you can still match.”

Stage 2 — retrieval

Given a question, the retriever scores every chunk and returns the top-k as source locations: (file_path, first_character_index, last_character_index). Two requirements in the subject deserve emphasis because they cause most silent failures:

- **Path exactness.** file_path is compared *verbatim* against the ground truth, e.g. data/raw/vllm-0.10.1/docs/features/lora.md. Store paths relative to the project root from day one; an absolute path or a different prefix scores zero forever.
- **Span width.** No retrieved source may exceed 2000 characters (max_context_length): one over-long source invalidates your entire output file. Enforce the invariant in the chunker *and* assert it again at serialization.

Retrieval must work in two modes: a single ad-hoc query (the search command) and batch mode over a JSON dataset of questions (search_dataset), the mode the moulinette consumes. The 90-seconds-for-200-questions budget is generous for an in-memory lexical index (typical: milliseconds per query) but rules out anything that reloads the index or re-tokenizes the corpus per question.

Stage 3 — augmentation

Augmentation is the neglected middle child of RAG, and where small-model pipelines are won or lost. You have k retrieved chunks, a token budget, and a model that is easily distracted. The questions to answer:

- **How many chunks to include?** k=10 maximizes recall for the grader, but the best *answers* often come from feeding only the top 3–5 chunks: fewer distractors, less dilution of attention. Nothing forces the generation prompt to use all k retrieved sources — the output JSON reports what you retrieved; the prompt is your choice.
- **In what order?** LLMs attend more reliably to the beginning and end of the context (“lost in the middle”). Put the best chunk first (or first and last).
- **With what framing?** Label each snippet with its file path — the model can then say “according to docs/features/lora.md ...”, and path names themselves carry signal. Instruct the model to answer only from the provided context.
- **Within what budget?** Compute an explicit character/token budget and truncate whole snippets (never mid-snippet if avoidable) to fit it.

Stage 4 — generation

Finally the model reads the assembled prompt and produces the answer. With Qwen3-0.6B, expect competent extraction and paraphrase of clearly relevant context, and expect failure on multi-hop synthesis across many snippets. The grading rubric anticipates this: retrieval quality, grounding, and prompt strategy dominate; “a valid answer is coherent and mostly grounded ... even if partially incomplete due to base-model limitations.” Chapter 12 covers prompt construction in depth.

The data contracts between stages

The subject formalizes inter-stage data as pydantic models — `MinimalSource`, `UnansweredQuestion`, `AnsweredQuestion`, `RagDataset`, `MinimalSearchResults`, `MinimalAnswer`, `StudentSearchResults`, `StudentSearchResultsAndAnswer`. Treat these as the pipeline's type system: indexing consumes files and produces the index; `search_dataset` consumes `RagDataset` and produces `StudentSearchResults`; `answer_dataset` consumes `StudentSearchResults` and produces `StudentSearchResultsAndAnswer`; the `moulinette` consumes your `StudentSearchResults` plus the ground-truth `RagDataset`. Drawing this dataflow on the whiteboard, unprompted, is the single fastest way to convince an evaluator you own the architecture.

Note — RAG variants you may be asked about at the defense, in one line each: **naïve RAG** — what you are building (index, retrieve, stuff, generate); **re-ranked RAG** — a second, more expensive model re-orders the top candidates; **hybrid RAG** — fuse lexical and semantic rankings (bonus 2); **agentic RAG** — the model decides when and what to search, possibly iteratively (this is where function calling from `call_me_maybe` meets RAG).

CHAPTER 5

Information Retrieval Fundamentals

Sixty years of search-engine science compressed into the vocabulary you need for this project.

Information Retrieval (IR) is the discipline of finding, in a large collection, the items relevant to an information need expressed as a query. Web search is IR; grep is (degenerate) IR; your retriever is textbook IR. The field's core insight, refined since the 1960s, is that *relevance can be usefully approximated by term statistics* — you do not need to understand a document to rank it.

The bag-of-words model

Lexical IR represents both documents and queries as **bags of words**: multisets of terms, ignoring order and grammar. “How does vLLM schedule requests?” becomes {how, does, vllm, schedule, requests} — or after normalization (Chapter 8), {vllm, schedul, request}. The representation is brutally lossy and astonishingly effective: for locating *where* something is discussed (rather than reasoning about it), word choice alone carries most of the signal. It also fails in a specific, predictable way—

The vocabulary mismatch problem

A question and its answering document may use different words for the same concept: the question says “GPU memory fraction”, the code says `gpu_memory_utilization`; the question says “speed up token generation”, the docs say “decoding throughput”. This is **vocabulary mismatch**, identified by Furnas et al. in 1987, and it is *the* fundamental limitation of lexical retrieval. Every recall point you lose on this project will trace back either to a chunking mistake or to vocabulary mismatch. The classical mitigations, all of which you will use:

- **Normalization**: lowercase, strip punctuation — “Scheduler” matches “scheduler”.
- **Identifier splitting**: `gpu_memory_utilization` -> `gpu, memory, utilization` — now the paraphrase “GPU memory” hits it. For a codebase corpus this single transformation is worth more than any other tuning knob.
- **Stemming**: `scheduling, scheduled, scheduler` -> `schedul`; a modest, cheap win on natural-language docs.
- **Keeping both forms**: emit the original token *and* its split parts, so verbatim identifier queries still match exactly (the subject's hint about “paraphrase an idea or quote an identifier verbatim” is telling you precisely this).
- **Dense/semantic retrieval** (bonus 1): embed meaning into vectors so that synonyms land close together — the modern answer to vocabulary mismatch.

The inverted index

The data structure that makes search fast is the **inverted index**: a map from each term to its **postings list** — the identifiers of the chunks that contain it (plus per-chunk term counts). “Inverted” because it flips the natural file->words direction into words->files.

An inverted index, conceptually

```
term          ->  postings (chunk_id : term_frequency)
-----
attention     ->  {12: 3, 507: 1, 1033: 7, 2210: 2, ...}
paged         ->  {507: 4, 1033: 2, ...}
scheduler     ->  {88: 5, 91: 2, 2210: 6, ...}
vllm          ->  {0: 1, 3: 2, 12: 1, ... (almost everywhere)}
```

To score a query you only touch the postings lists of the query's terms — for a 5-term query over a 30,000-chunk corpus, perhaps a few thousand postings rather than 30,000 full documents. Python dictionaries make a perfectly adequate implementation at this scale; Volume 2 builds one in ~40 lines, and it answers queries in single-digit milliseconds.

Zipf's law, stopwords, and why frequency must be discounted

Term frequencies in natural language follow **Zipf's law**: the r -th most frequent word has frequency roughly proportional to $1/r$. A handful of words (“the”, “of”, “to”) account for a huge share of all tokens while carrying almost no discriminative power, and a long tail of rare words carries almost all of it. Two practical consequences: (1) dropping a few dozen **stopwords** shrinks the index and removes noise — but keep the list short and domain-aware, because in a codebase corpus words like “if” or “while” may appear inside meaningful identifiers, and aggressive stoplists hurt; (2) any sane scoring function must weight terms by their rarity — which is exactly what IDF does, and where the next chapter begins.

Precision, recall, and ranked retrieval

Two classical measures describe a retrieval result set. **Precision**: what fraction of what I returned is relevant? **Recall**: what fraction of what is relevant did I return? They trade off against each other — return everything and recall is 1 while precision collapses; return one perfect hit and precision is 1 while recall may be tiny. Modern systems sidestep the fixed-set framing by returning a **ranking** and evaluating it at cutoffs: precision@k, recall@k. Your grader uses recall@k, defined per question as the share of that question's ground-truth sources that appear (by file + span overlap) in your top k; the full machinery, including the IoU ≥ 0.05 overlap rule and worked examples, is in Chapter 11 and again, operationally, in Volume 3.

Note — A subtlety worth quoting at the defense: recall@k does *not* reward ordering within the top k. Recall@5 is identical whether the right chunk is rank 1 or rank 5. But recall@1 $\leq$ recall@3 $\leq$ recall@5 $\leq$ recall@10 always, and the moulinette prints all four — a healthy system shows a rising staircase, and a large gap between recall@5 and recall@10 means your scoring is nearly right but your ordering is weak.

CHAPTER 6

TF-IDF from Scratch

The first weighting scheme that worked — and the conceptual ancestor of everything you will implement.

The subject lets you choose TF-IDF or BM25. Even if you implement BM25 (and you should — Chapter 7 explains why), you must understand TF-IDF, because BM25 *is* TF-IDF with two corrections, and because “explain TF-IDF” is the most probable theory question at your defense.

The two intuitions

1. **Term Frequency (TF):** a document that mentions “scheduler” seven times is more likely to be about schedulers than one that mentions it once. Repetition signals aboutness.
2. **Inverse Document Frequency (IDF):** a match on “paged” (rare, appears in few documents) is far more informative than a match on “vllm” (appears everywhere in this corpus). Rarity signals discriminative power.

TF-IDF simply multiplies the two, scoring term t in document d within corpus D :

Classic TF-IDF

$\text{tf_idf}(t, d) = \text{tf}(t, d) * \text{idf}(t)$

$\text{tf}(t, d)$	= count of t in d	(raw form)
$\text{idf}(t)$	= $\log(N / \text{df}(t))$	(classic form)

N = number of documents in the corpus

$\text{df}(t)$ = number of documents containing t at least once

A document's score for a multi-term query is the sum over query terms. Common refinements, each of which you should be able to justify:

- **Log-scaled TF:** $\text{tf} = 1 + \log(\text{count})$. The 10th occurrence of a term says much less than the 1st; raw counts over-reward repetitive documents. This is a first, crude form of the saturation idea BM25 makes explicit.
- **Smoothed IDF:** $\text{idf} = \log(N / (1 + \text{df}))$ or $\log((1+N)/(1+\text{df})) + 1$ (scikit-learn's default), avoiding division by zero and taming terms that occur in every document.
- **Cosine normalization:** treat documents as vectors in vocabulary space and divide by their Euclidean norm, so long documents do not win merely by containing more words. In vector form, TF-IDF retrieval is: embed query and documents as sparse TF-IDF vectors, rank by cosine similarity.

A worked example, by hand

Take a toy corpus of $N = 4$ “documents” (chunks) and the query “*paged attention memory*”. Document frequencies: $df(paged)=1$, $df(attention)=3$, $df(memory)=2$. Using $idf = \log(N/df)$ with natural log:

Worked TF-IDF scoring (raw TF, ln IDF)

```
idf(paged)      = ln(4/1) = 1.386
idf(attention)  = ln(4/3) = 0.288
idf(memory)     = ln(4/2) = 0.693
```

doc	tf in doc:			weighted contributions:			SCORE
	paged	attention	memory	paged	attention	memory	
D1	0	4	0	0	$4 \times 0.288 = 1.15$	0	1.15
D2	2	1	1	$2 \times 1.386 = 2.77$	0.288	0.693	3.75
D3	0	1	3	0	0.288	$3 \times 0.693 = 2.08$	2.37
D4	0	0	0	0	0	0	0.00

D2 wins decisively — not because it has the most matches (D1 has four occurrences of “attention”), but because it contains the *rare* term “paged” twice. This is IDF doing its job: one occurrence of a rare, specific term outweighs many occurrences of a common one. Notice also a pathology: D1’s four repetitions of “attention” earn it four full increments of a low weight — linear growth with no ceiling. A 2000-character chunk that happens to repeat “attention” 40 times would score ten times higher still, which clearly does not make it ten times more relevant. Fixing this pathology is exactly the first of BM25’s two corrections.

TF-IDF as a vector space

Formally, fix a vocabulary V ; each document becomes a sparse vector in $\mathbb{R}^{|V|}$ whose t -th coordinate is $tf_idf(t, d)$; similarity between query vector q and document vector d is their cosine: $\cos(q, d) = (q \cdot d) / (|q||d|)$. This geometric view matters for two reasons. First, it is what scikit-learn’s `TfidfVectorizer + cosine_similarity` implement, if you choose the library route. Second, it is the conceptual bridge to embeddings (Chapter 10): dense retrieval replaces the sparse, vocabulary-indexed axes with learned, meaning-indexed axes, keeping the same “rank by vector similarity” skeleton. TF-IDF is an embedding — just a very sparse, very literal one.

Where TF-IDF falls short

- No term-frequency saturation: linear TF over-rewards repetition (log-TF only softens it).
- No principled length handling: cosine normalization punishes all long documents equally, whether they are verbose or genuinely rich; short-chunk corpora are mostly fine, mixed-length corpora suffer.
- IDF unbounded and unsmoothed in the classic form.
- Still purely lexical: synonyms score zero (vocabulary mismatch untouched).

Each of the first three shortcomings is individually addressed by BM25’s design, which is why, sixty years on, BM25 — not raw TF-IDF — is the default lexical ranking function in Lucene, Elasticsearch, and every serious search system. On to it.

CHAPTER 7

BM25: The Industry Workhorse

Two fixes to TF-IDF, derived from a probabilistic argument, tunable with two knobs — and worth several recall points.

BM25 (“Best Matching 25” — literally the 25th ranking function tried in the Okapi system at City University London, Robertson & Walker, 1994) emerges from the **probabilistic relevance framework**: rank documents by their estimated probability of relevance to the query. You do not need the full derivation, but you should know that BM25 is not an arbitrary formula — each piece has a probabilistic justification, and the evaluator may ask what each part does.

The formula

BM25 (the Okapi formulation used by rank-bm25, Lucene et al.)

score(q, d) = SUM over terms t in q of:

$$\text{idf}(t) * \frac{\text{tf}(t,d) * (k1 + 1)}{\text{tf}(t,d) + k1 * (1 - b + b * |d| / \text{avgdl})}$$

with the Robertson-Sparck Jones smoothed IDF:

$$\text{idf}(t) = \ln((N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5) + 1)$$

tf(t,d) : occurrences of t in chunk d
 |d| : length of chunk d in tokens
 avgdl : average chunk length over the corpus
 N, df : as before
 k1, b : free parameters (defaults k1=1.5, b=0.75)

Fix #1 — term-frequency saturation (k1)

Look at the TF factor alone: $\text{tf} \cdot (k1+1) / (\text{tf} + k1 \cdot (...))$. As tf grows, this ratio does not grow linearly — it asymptotically approaches (k1+1). The first occurrence of a term moves the score a lot; the fifth, a little; the fiftieth, almost nothing. Concretely, with k1 = 1.5 and a chunk of average length (so the length factor is 1):

Saturation: $\text{tf} \cdot (k1+1) / (\text{tf} + k1)$, k1=1.5 — diminishing returns are built in

tf :	1	2	3	5	10	50
gain:	1.000	1.429	1.667	1.923	2.174	2.427

(max -> 2.5)

k1 controls how fast saturation kicks in. k1 = 0 ignores term frequency entirely (every match counts once — pure presence/absence); large k1 approaches linear TF-IDF behaviour. The default 1.5 (rank-bm25 uses 1.5; Lucene 1.2) is a robust starting point. For *code* chunks, where an identifier may repeat dozens of times in a loop-heavy file without being “more about” that identifier, slightly lower k1 (1.0–1.2) often helps; Volume 3’s tuning chapter measures this.

Fix #2 — pivoted length normalization (b)

The denominator's $k1 \cdot (1 - b + b \cdot |d| / \text{avgdl})$ term rescales the saturation point by chunk length. A chunk twice the average length needs roughly twice the term occurrences to reach the same score — longer documents get more chances to contain any term, so their counts are discounted. **b interpolates between no normalization (b=0) and full proportional normalization (b=1)**; 0.75 is the time-honoured default. Two project-specific remarks: because your chunker caps every chunk at 2000 characters, your length distribution is already fairly tight, so b matters less than in a corpus of wildly varying documents; but b still matters at the margins because code chunks and prose chunks tokenize to different lengths, and Volume 3 shows measurable movement from tuning b in [0.3, 0.9].

The smoothed IDF

BM25's IDF, $\ln((N - df + 0.5) / (df + 0.5) + 1)$, behaves like $\ln(N/df)$ for rare terms but is smoothed at both ends: the +0.5s (a Bayesian half-count) prevent infinities and wild swings for df near 0 or N, and the outer +1 (a Lucene refinement adopted by rank-bm25) keeps the value positive even for terms in more than half the documents. Without that +1, a term appearing in most chunks — think “vllm” or “def” in this corpus — would get a *negative* weight, actively punishing documents for containing the query term. Knowing that detail is defense gold.

Worked example

Same toy corpus as Chapter 6 (N=4; df: paged=1, attention=3, memory=2), query “paged attention memory”, k1=1.5, b=0.75. Chunk token lengths: |D1|=100, |D2|=120, |D3|=80, |D4|=100, so avgdl=100.

Full BM25 computation by hand — bring this to the defense

IDFs:

```
idf(paged)      = ln((4-1+0.5)/(1+0.5) + 1) = ln(3.333) = 1.204
idf(attention)  = ln((4-3+0.5)/(3+0.5) + 1) = ln(1.429) = 0.357
idf(memory)     = ln((4-2+0.5)/(2+0.5) + 1) = ln(2.000) = 0.693
```

D1 (|d|=100, len factor = $1.5 \cdot (0.25 + 0.75 \cdot 1.0) = 1.5$):

```
attention tf=4: 0.357 * (4*2.5)/(4+1.5)      = 0.357*1.818 = 0.649
SCORE(D1) = 0.649
```

D2 (|d|=120, len factor = $1.5 \cdot (0.25 + 0.75 \cdot 1.2) = 1.725$):

```
paged      tf=2: 1.204 * (2*2.5)/(2+1.725)    = 1.204*1.342 = 1.616
attention  tf=1: 0.357 * (1*2.5)/(1+1.725)    = 0.357*0.917 = 0.327
memory     tf=1: 0.693 * (1*2.5)/(1+1.725)    = 0.693*0.917 = 0.636
SCORE(D2) = 2.579
```

D3 (|d|=80, len factor = $1.5 \cdot (0.25 + 0.75 \cdot 0.8) = 1.275$):

```
attention tf=1: 0.357 * 2.5/(1+1.275)        = 0.357*1.099 = 0.392
memory     tf=3: 0.693 * (3*2.5)/(3+1.275)    = 0.693*1.754 = 1.216
SCORE(D3) = 1.608
```

Ranking: D2 (2.58) > D3 (1.61) > D1 (0.65) > D4 (0)

Compare with the TF-IDF ranking from Chapter 6 (D2 3.75, D3 2.37, D1 1.15): the order happens to agree here, but look at the *ratios*. Under TF-IDF, D1's four “attention”s earned 1.15 — under BM25

only 0.649, because saturation capped the repetition and D1's exactly-average length gave no discount. On a real corpus with pathological chunks (long tables, repeated boilerplate, vendored lists), these corrections routinely change the top-5 — and recall@5 with them.

Implementing vs importing

You may implement BM25 yourself (Volume 2 does, in about sixty lines with an inverted index) or use a library (rank-bm25's BM25Okapi is the de-facto standard; scikit-learn has TF-IDF only). The subject allows any library. The argument for hand-rolling: total control over the tokenizer (critical, see next chapter), zero dependency risk, effortless persistence of exactly the statistics you need, and unbeatable defense credibility. The argument for rank-bm25: fewer lines. The reference implementation hand-rolls, and Volume 2 explains every line; if you import instead, be ready to write the formula on paper anyway.

Note — Variants worth naming if asked: **BM25L** and **BM25+** adjust the lower end of length normalization so very long documents are not over-penalized; **BM25F** scores multiple fields (title, body, path) with different weights — conceptually what you do if you boost matches in file paths or Markdown headings, a trick Volume 3 uses.

CHAPTER 8

Tokenization and Normalization for Search

Questions and source code rarely use the same words. What you keep at indexing time decides what you can still match. — the subject

Between raw text and the inverted index stands the **analyzer**: the function that turns a string into the list of terms that will actually be indexed and matched. It is the least glamorous component of your system and, for a codebase corpus, the single highest-leverage one. Two retrievers with identical BM25 implementations and different analyzers can differ by 20 recall points. This chapter designs the analyzer; Volume 2 ships it as ~30 lines of code.

The golden rule: one analyzer, two call sites

Whatever transformation you apply to documents at indexing time, you must apply *identically* to queries at search time. Index “Scheduler” lowercased but search for raw “Scheduler”, and the term never matches its own postings list. Make the analyzer a single pure function imported by both the indexer and the retriever; never write the logic twice. Most inexplicable recall bugs in student RAG projects are analyzer asymmetries.

Stage 1 — token extraction

Split raw text into candidate tokens. For mixed prose-and-code content, a regex over word characters works well:

Token extraction — deliberately keeps underscores, drops other punctuation

```
import re
TOKEN_RE = re.compile(r"[A-Za-z0-9_]+")
tokens = TOKEN_RE.findall(text)
```

Keeping the underscore is deliberate: it preserves `snake_case` identifiers as single tokens for the moment, letting the next stage split them *while also keeping the original*. Dropping other punctuation folds “`schedule()`” and “`schedule`” together — which is what you want.

Stage 2 — identifier splitting

Source code concentrates meaning in compound identifiers: `gpu_memory_utilization`, `PagedAttention`, `max_num_batched_tokens`, `LLMEngine`. A question asks about “GPU memory utilization” in three words; the code says it in one token. Splitting compounds bridges the gap:

- **snake_case**: split on underscores -> `gpu`, `memory`, `utilization`.

- **camelCase / PascalCase:** split at lower-to-upper boundaries -> paged, attention. Handle acronym runs: LLMEngine -> llm, engine (boundary between a final capital and a following lowercase letter).
- **digit boundaries:** fp8, v0, qwen3 — splitting digits from letters is optional; the reference keeps alphanumeric runs intact (splitting “fp8” into “fp”, “8” creates noise).

Identifier splitting: snake_case, camelCase, PascalCase, ACRONYMCase

```
CAMEL_RE = re.compile(
    r"(?<=[a-z0-9])(?=[A-Z])|(?<=[A-Z])(?=[A-Z][a-z])"
)
```

```
def split_identifier(tok: str) -> list[str]:
    parts: list[str] = []
    for piece in tok.split("_"):
        if piece:
            parts.extend(CAMEL_RE.split(piece))
    return [p for p in parts if p]
```

Emit both the whole and the parts. If the analyzer outputs [gpu_memory_utilization, gpu, memory, utilization], then a paraphrasing question matches through the parts while a question quoting the identifier verbatim matches the intact token with full IDF force (the intact token is rare, so its IDF is high — exactly the behaviour you want). This directly implements the subject's hint that a question “may paraphrase an idea or quote an identifier verbatim”, and it is the most important paragraph in this chapter.

Stage 3 — case folding and stopwords

Lowercase everything after splitting (splitting first, because case carries the camelCase boundaries). Then optionally drop stopwords. For this corpus a *short* English stoplist (~30 words: the, a, of, to, is, in, and...) is safe and helpful for docs questions; a long aggressive stoplist is risky because query words like “how”, “not”, “while” can carry meaning in code contexts. Numbers: keep them — version strings (0.10.1) and sizes (8192) appear in real questions.

Stage 4 — stemming (optional, measurable)

A **stemmer** maps inflected forms to a common stem: scheduling, scheduled, schedules -> schedul. The Porter/Snowball stemmers (available via the nltk or snowballstemmer packages) are the classics. Stemming typically buys 1–3 recall points on natural-language docs questions and is near-neutral on code. It also has costs: stems are unreadable in debug output, over-stemming merges distinct words (university/universe), and it adds a dependency. The defensible choices are (a) no stemming, analyzer stays trivially explainable, or (b) Snowball stemming, +1–3 points, measured before/after with your evaluate command. What is not defensible is cargo-culting either choice; run the experiment and put the numbers in your README's performance analysis section.

What the analyzer must NOT do

- Do not run the analyzer on file *contents* only and forget that paths carry signal — “docs/features/lora.md” analyzed as [docs, features, lora, md] is a free relevance boost when the question mentions LoRA (Volume 3 implements path-term boosting).
- Do not destroy the character offsets. The analyzer feeds scoring only; the chunk’s (first_character_index, last_character_index) always refer to the *original raw file text*, untouched. Never index a “cleaned” version of the file and then report offsets into it — the moulinette measures overlap against the raw file.
- Do not make it slow. The analyzer runs over $\sim 10^7$ characters at indexing time; keep it regex-based and allocation-light. Precompile patterns at module load.

Transformation	Docs recall	Code recall	Verdict
Lowercasing	required	required	always
Identifier splitting (keep both)	helps	large gain	always
Short stoplist (~30 words)	helps	neutral	recommended
Snowball stemming	+1-3 pts	neutral	measure it
Aggressive stoplist (100+ words)	risky	harmful	avoid
Splitting letter-digit runs (fp8)	neutral	noisy	avoid

CHAPTER 9

Chunking Theory

A Python file and a Markdown page do not break apart the same way. — the subject, mandating two strategies

Chunking decides what a “retrievable unit” is. It is the most consequential design decision in the entire project: recall is computed over character-range overlap with ground-truth spans, so where you cut directly determines whether a hit is even possible. This chapter builds the theory; Volume 2 implements both mandated strategies.

Why chunk at all?

- **Scoring granularity.** BM25 over whole files would tell you “somewhere in this 4,000-line file” — useless for a 2000-character answer span, and length normalization degrades over wildly varying document sizes.
- **The grader demands it.** Every reported source must be at most 2000 characters wide (`max_context_length`); a single wider span invalidates the whole output file. Whole files are simply not a legal answer.
- **The generator demands it.** Chunks are what you paste into Qwen's context; they must be small enough that several fit in the budget, and self-contained enough to be understood in isolation.

The size trade-off

Smaller chunks are more precise (the matched span is mostly the relevant text; BM25 statistics are sharper because the term density of the topic is higher) but riskier (an answer that spans a paragraph boundary may be split across two chunks, each scoring too weakly to surface; context becomes fragmentary). Larger chunks are safer containers (more likely to fully contain a ground-truth span) but dilute term statistics and waste context-window budget on padding. The subject fixes the ceiling at 2000 characters and invites you to explore below it (“Smaller chunks are fine; report the effect on your `recall@k`”). Two forces specific to this project push toward *generous* chunks near the ceiling: the IoU bar is a low 0.05 (a big chunk overlapping a small true span still counts — see Chapter 11 for the exact geometry), and ground-truth spans were themselves authored under the 2000-character convention. The reference implementation defaults to ~1800-character targets with the hard 2000 cap and measures alternatives; you should run your own sweep (Volume 3 shows how) and report it in your README.

Overlap

Fixed-size chunkers often add **overlap** (e.g. 200 characters repeated between consecutive chunks) so that content near a boundary appears whole in at least one chunk. Overlap costs index size and

can put two near-duplicate chunks into your top-k (wasting slots — consider deduplicating near-identical spans at retrieval time if you use it). Structure-aware chunking reduces the need for overlap because boundaries fall where meaning naturally breaks; the reference uses structural boundaries plus modest overlap only in the fallback fixed-size path.

Strategy A — Markdown and prose

Markdown declares its own structure; use it. The natural boundary hierarchy:

1. Split at **headings** (lines matching `^{1,6}`) — a section is the natural unit of documentation meaning; a heading plus its body answers a question together. Keep the heading line *with* its section: the heading's words (“LoRA adapters”, “Quantization”) are often exactly the query's words.
2. If a section exceeds max size, split at **blank lines** (paragraph boundaries), greedily packing paragraphs into chunks up to the limit.
3. Never split inside a **fenced code block** (`` ... ``) if avoidable — half a code example matches nothing well. Track fence state while scanning lines.
4. If a single paragraph or fence still exceeds the limit (it happens: long tables, minified content), fall back to a hard character split with a small overlap. Correctness (the 2000 cap) beats elegance.

This same strategy, minus the heading logic, degrades gracefully to plain-text chunking for .txt/.rst/config files: split on blank lines, pack to size, hard cap as last resort. That is why the subject phrases it as “Markdown / text chunking” — one strategy family.

Strategy B — Python code

Code's meaning units are syntactic: functions, classes, methods. Python's standard-library **ast** module parses a file and reports, for every top-level definition, its start and end line (`end_lineno`, available since Python 3.8). The chunking algorithm:

1. Parse the file with `ast.parse`. On `SyntaxError` (rare but real — Python 2 relics, template files), fall back to the text chunker.
2. Walk top-level nodes: each function/class becomes a candidate chunk spanning its source lines, *including decorators* (`decorator_list[0].lineno` may precede `node.lineno`) and ideally the preceding comment block.
3. Class bodies larger than the cap: descend one level and chunk per method, keeping the class header line attached to each method chunk for context (so a method chunk reads “class Scheduler: ... def schedule(...)”).
4. Module-level statements between definitions (imports, constants, argparse setup) are gathered into interstitial chunks.
5. Any candidate still over the cap (a 300-line function) gets the hard split.

Line numbers become character offsets by precomputing a line-offset table (cumulative lengths of `file.splitlines(keepends=True)`); Volume 2 gives the exact code with its off-by-one traps dissected. The payoff of AST chunking is that a question about “how does the scheduler decide preemption” retrieves the *whole* relevant method — docstring, signature, body — as one coherent, high-scoring unit, rather than a window that starts mid-loop.

Watch out — Character indices are in **characters of the decoded text**, not bytes. Read files as UTF-8 with `errors='replace'` (or `'ignore'` — but pick one and be consistent between indexing and any later re-reading), and never mix byte offsets (from `file.seek` or `regex` on bytes) with str indices. A corpus file with a single emoji or accented name will silently shift every subsequent offset if you conflate the two.

Invariants your chunker must guarantee

1. `last_character_index - first_character_index <= 2000` for every chunk, no exceptions, enforced by construction and asserted in tests.
2. `text[first:last]` (Python slice semantics on the raw decoded file) reproduces exactly the chunk's text — the offsets are the ground truth, the text is a cache. Decide once whether `last_character_index` is exclusive (Python-style; the reference does this, matching the subject's examples where spans like `(0, 500)` appear) and stay consistent.
3. Chunks cover all indexable content: no gaps that could swallow a ground-truth span. Whitespace-only gaps are acceptable.
4. Deterministic: same file, same parameters, same chunks — reindexing must be reproducible for debugging and incremental indexing (bonus 3).

CHAPTER 10

Dense Retrieval and Embeddings

Theory you need for bonus 1 and 2 — and for the inevitable defense question: “how would you beat vocabulary mismatch?”

Lexical retrieval matches strings; **dense retrieval** matches meanings. A neural **embedding model** maps any text to a vector (384 dimensions for the classic all-MiniLM-L6-v2) such that semantically similar texts land close together. Retrieval becomes geometry: embed all chunks offline, embed the query at search time, return the k nearest neighbours by cosine similarity.

How embedding models are trained

Sentence embedders like MiniLM are **bi-encoders**: a small transformer encodes each text independently into one vector (mean-pooled token states, L2-normalized). They are trained with **contrastive learning** over massive pair datasets (question/answer pairs, duplicate questions, citation pairs): pull genuine pairs together, push random in-batch negatives apart (the InfoNCE / multiple-negatives-ranking loss). The result: “How do I limit GPU memory?” and “gpu_memory_utilization controls the fraction of GPU memory allocated” end up close in the space even though they share almost no words. That is vocabulary mismatch, solved — statistically, not perfectly.

Cosine similarity and normalization

Dense scoring is linear algebra; numpy does a corpus in milliseconds

$$\cos(q, d) = (q \cdot d) / (|q| * |d|)$$

If both vectors are pre-normalized to unit length:

$$\cos(q, d) = q \cdot d \quad (\text{a single dot product})$$

All-chunks scoring = one matrix-vector product:

```
scores = E @ q      # E: (num_chunks, 384) float32 matrix
```

At this project's scale (tens of thousands of chunks) you do not need a vector database or approximate-nearest-neighbour indexes (FAISS, HNSW): brute-force numpy over a preloaded float32 matrix meets the 90 s / 200 questions budget with two orders of magnitude to spare. Say this at the defense when asked why you didn't use a vector DB — right-sizing is engineering.

Lexical vs dense: complementary failure modes

	Lexical (BM25)	Dense (MiniLM)
Exact identifiers, error strings	excellent — exact match, high IDF	mediocre — subwords blur rare tokens
Paraphrase, synonyms	blind (vocabulary mismatch)	excellent

	Lexical (BM25)	Dense (MiniLM)
Out-of-domain jargon	fine if strings match	degrades — training distribution matters
Index cost	seconds, tiny	minutes on CPU (one forward pass per chunk), ~50 MB of vectors
Explainability	perfect — point at matching terms	opaque
Additional dependency	none	sentence-transformers + torch

A codebase corpus is the best case for lexical and a hard case for a general-purpose embedder — questions quote identifiers, and MiniLM was not trained on vLLM jargon. This is why the subject makes lexical mandatory and dense a bonus, and why **hybrid** retrieval (bonus 2) is the real prize: each method covers the other's blind spot.

Fusing rankings: Reciprocal Rank Fusion

The cleanest way to combine two rankings with incomparable score scales is to ignore the scores and fuse the *rank*s. Reciprocal Rank Fusion (RRF, Cormack et al. 2009):

Reciprocal Rank Fusion — robust, parameter-light, no score calibration

$RRF(d) = \text{SUM over each ranking } r \text{ of: } 1 / (K + \text{rank}_r(d))$

$K = 60$ by convention; rank starts at 1; a document absent from one ranking simply contributes nothing from it.

Why it works: $1/(K+\text{rank})$ decays smoothly, so agreement between rankers dominates (a chunk ranked 3rd by both beats one ranked 1st by one and 200th by the other), and K damps the influence of any single ranker's top ranks. The alternative — z-score-normalizing BM25 and cosine scores and taking a weighted sum — works too but adds a weight to tune and a calibration to defend. Volume 3 implements RRF in a dozen lines.

Note — Cross-encoders and re-ranking, for completeness: a **cross-encoder** feeds query and document *jointly* through a transformer and outputs a relevance score — far more accurate than a bi-encoder, far too slow to run over a corpus, so it is used to re-rank a candidate top-50. That is the standard third stage in industrial search (retrieve -> fuse -> re-rank). Not required here, but knowing the pattern rounds out your defense answers.

CHAPTER 11

Evaluation: Recall@k, IoU, and Friends

You cannot improve what you cannot measure — and you cannot pass what you measure differently from the grader.

The moulinette computes one family of numbers — recall@k for k in {1, 3, 5, 10} — and applies two thresholds: docs $\geq 80\%$ and code $\geq 50\%$, both at k=5. This chapter derives the metric exactly, because your own evaluate command must reproduce it: tuning against a metric that differs from the grader's is worse than not measuring at all.

The matching predicate: same file + IoU ≥ 0.05

A ground-truth source g counts as **found** if at least one of your top-k retrieved sources r satisfies both:

1. `r.file_path == g.file_path`, compared as exact strings (this is why the data/raw/vllm-0.10.1/... prefix must be reproduced verbatim);
2. $\text{IoU}(r, g) \geq 0.05$, where IoU (intersection over union) of the character intervals is $|r \cap g| / |r \cup g|$.

Interval IoU in four lines — the heart of the grader

```
def iou(a_start, a_end, b_start, b_end):
    inter = max(0, min(a_end, b_end) - max(a_start, b_start))
    union = (a_end - a_start) + (b_end - b_start) - inter
    return inter / union if union > 0 else 0.0
```

Feel the geometry with numbers. Your 2000-char chunk $r = (0, 2000)$ versus a true span $g = (1500, 1900)$: intersection 400, union $2000 + 400 - 400 = 2000$, IoU 0.20 — found, comfortably. Same chunk versus $g = (1990, 2390)$: intersection 10, union 2390, IoU 0.004 — missed, even though you technically overlap. Now two equal 2000-char spans offset by 1905: intersection 95, union 3905, IoU 0.024 — missed. The lesson: the 0.05 bar is low but not zero; grazing contact at a chunk edge does not count, and this is precisely the failure mode that overlap or structure-aware boundaries exist to prevent. Worst case for equal 2000-char spans: you need roughly ≥ 190 characters of overlap ($190/3810 \sim 0.0499\dots$) — in practice, “about 10% of the chunk must genuinely cover the target”.

Recall@k, exactly

Recall@k as the moulinette computes it

For one question q with ground-truth sources $G(q)$:

```
found(q, k) = |{ g in G(q) : some r in top-k results matches g }|
recall@k(q) = found(q, k) / |G(q)|
```

Dataset score = mean over questions of recall@k(q) (macro average)

Note the macro average: every question weighs equally, whether it has one ground-truth source or five. A question with 4 sources of which you found 3 contributes 0.75; a one-source question is all-or-nothing. Consequences for strategy: multi-source questions reward retrieving *diverse* locations (if all 5 of your top-5 pile onto the same file region, you can match at most one ground-truth source there), and one-source questions reward precision at the very top. Volume 3 exploits both observations when tuning.

Why recall (and not precision) is the graded metric

In a RAG pipeline the retriever's job is to make sure the evidence is *present* in the context; the generator tolerates some irrelevant passengers. A missed source is unrecoverable — the model cannot cite what it never saw; an extra source is merely dilution. Hence recall@k, with k=5 as the grading point and k=10 as what you actually return (--k 10 in the reference commands, giving the generator more to work with while grading looks at the top of the ranking... careful: re-read that). Actually, precisely because recall@5 is computed over your *first five* results, ordering between positions 5 and 10 is invisible to the pass threshold but recall@10 still gets printed — a diagnostic gift: a big recall@10 - recall@5 gap says “right neighbourhood, wrong order”, pointing you at scoring (k1, b, boosts) rather than chunking.

The metrics you will not be graded on (but should know)

Metric	One-line definition	What it adds
Precision@k	relevant results in top-k / k	penalizes junk in the list
MRR	mean of 1/rank of the first relevant result	how high is the first hit
MAP	mean average precision over recall levels	order-sensitive overall quality
nDCG@k	discounted gain vs ideal ordering, graded relevance	handles 'somewhat relevant' judgements
End-to-end answer quality	human or LLM-judged faithfulness/completeness	what users actually experience

If the evaluator asks “is recall@5 a good metric?”, the strong answer is: it measures exactly what the generator needs (evidence presence), it is cheap and deterministic, but it is blind to ordering inside the top-k, blind to junk, and its 0.05 IoU makes it lenient about span quality — which is why production systems track nDCG and answer-level metrics alongside. Then stop talking. Knowing the boundary of a metric is worth more than reciting five others.

CHAPTER 12

Grounded Generation with a 0.6B Model

The library is open, the right page is on the desk. Now make a very small librarian read it aloud correctly.

Generation is graded qualitatively — coherent, grounded, on point — with explicit tolerance for Qwen3-0.6B's limits. Your leverage is entirely in how you assemble the prompt and post-process the output. This chapter gives the theory and the concrete prompt design; Volume 2 wires it into code.

Know your instrument

- Qwen3-0.6B: 28 layers, hidden 1024, ~0.6B parameters, 32,768-token native context, ChatML template, optional thinking mode. Runs on CPU; expect roughly 5–20 tokens/s CPU generation depending on hardware — budget accordingly for answer_dataset over hundreds of questions (cap max_new_tokens; 256 is plenty for a grounded answer).
- Strengths at this size: extraction, paraphrase of present text, format compliance on short outputs.
- Weaknesses: multi-hop reasoning across snippets, arithmetic, long-output coherence, resisting a strongly-worded false premise in the question.

Prompt architecture for grounded QA

The prompt is a contract with three clauses — role, evidence, task — plus the discipline of putting *instructions before evidence and the question after both* (recency helps small models stay on task):

The grounded-QA prompt shape used by the reference implementation

SYSTEM:

You are a precise technical assistant answering questions about the vLLM codebase. Answer ONLY from the provided context snippets. If the context does not contain the answer, say so briefly. Cite file paths when relevant. Be concise: 2-6 sentences.

USER:

Context:

[Source 1] data/raw/vllm-0.10.1/docs/features/lora.md (chars 900-2650)
<snippet text>

[Source 2] data/raw/vllm-0.10.1/vllm/lora/request.py (chars 0-1400)
<snippet text>

Question: How do I load a LoRA adapter at request time?

Each element earns its place. The role line anchors register and domain. “Answer ONLY from the provided context” is the grounding instruction — small models follow it imperfectly, but measurably better than without. The escape hatch (“if the context does not contain the answer, say so”) reduces fabricated answers on retrieval misses — without it, the model *will* invent something, because

refusing is out-of-distribution. Labelled sources let the answer cite paths (nice for the grader's “grounded” criterion). The length cap fights the small-model tendency to pad with generic filler.

Context budgeting

With $k=10$ chunks of ≤ 2000 chars, raw context is $\leq 20,000$ chars $\sim 5\text{--}7\text{k}$ tokens: fits, but more is not better. Empirically with 0.6B-class models, answer quality peaks around 3–5 well-chosen snippets and degrades as marginal snippets dilute attention. The reference: feed the top 5 (by retrieval rank) into the prompt, truncate the total context at a fixed character budget ($\sim 8,000$ chars), dropping whole snippets from the tail rather than cutting one mid-sentence, while still *reporting* all k retrieved sources in the output JSON. The `retrieved_sources` field documents retrieval; the prompt is your editorial choice. Be able to state that distinction crisply.

Decoding and post-processing

- Disable thinking mode for dataset runs (`enable_thinking=False` in `apply_chat_template`): 3–10× faster, no trace to strip. Keep the defensive strip anyway: if `'</think>'` appears in the decoded output, keep only what follows it.
- Sampling per Qwen's non-thinking recommendation (temperature 0.7, top_p 0.8, top_k 20) or near-greedy with a repetition penalty; both are defensible — the former follows vendor guidance, the latter maximizes determinism. Pick one, know why.
- `max_new_tokens` ~ 256 ; trim trailing incomplete sentences; collapse whitespace; the answer field must be a clean single string in the JSON.
- Empty-context edge case: if retrieval returned nothing (empty query, $k=0$), generate no call at all — emit a fixed 'no relevant sources found' answer. Never crash, never hallucinate an answer with zero sources attached.

Hallucination under RAG: the failure taxonomy

Failure	Cause	Mitigation
Confident answer, wrong evidence	retrieval missed; model fell back on parametric memory	better retrieval; the escape-hatch instruction
Answer contradicts the snippet	model misread or overrode context	shorter context, put best snippet first, lower temperature
Answer beyond the snippet	model 'completed' partial evidence	grounding instruction; ask for citations
Generic non-answer	context present but question paraphrased oddly	include the question verbatim at the end of the prompt
Endless rambling / repetition	greedy decoding in thinking mode, no length cap	vendor sampling params, <code>max_new_tokens</code> , repetition penalty

During the defense, being able to produce this taxonomy — and point at which mitigation you implemented where — demonstrates exactly the “prompt strategy” competence the rubric rewards.

CHAPTER 13

Knowing the Corpus: vLLM

You are indexing a real, famous codebase. Ten minutes of orientation buys you better chunking, better debugging, and better defense answers.

vLLM is a high-throughput, open-source LLM inference and serving engine, originally from UC Berkeley's Sky Computing Lab, built around the **PagedAttention** idea: manage the KV cache (Chapter 2) like virtual memory, in fixed-size blocks, eliminating fragmentation and enabling far higher batch sizes. Version 0.10.1 — pinned in your `data/raw/` — is a large, professional repository, and the question datasets are written against it.

The map of the repository

Path	Contents	Retrieval relevance
<code>vllm/</code>	the Python package: engine, scheduler, attention, entrypoints (incl. the OpenAI-compatible server), lora, multimodal, distributed executors	code questions live here
<code>docs/</code>	MkDocs documentation: features, serving, models, getting started	docs questions live here
<code>csrc/</code>	C++/CUDA kernels	index as text or skip; few questions, heavy files — the reference indexes small ones as text
<code>examples/</code>	runnable Python + Markdown examples	moderate — index them
<code>benchmarks/</code>	benchmark scripts and docs	moderate
<code>tests/</code>	the test suite	low-to-moderate; large — index or exclude, but know the disk/time cost either way
<code>README.md</code> , <code>RELEASE.md</code> , ...	top-level prose	cheap to index, occasionally the ground truth

Vocabulary you will meet in questions and should recognize on sight: **PagedAttention** (block-based KV-cache management), **continuous batching** (new requests join a running batch between decode steps), **prefix caching** (reuse KV blocks of shared prompt prefixes), **speculative decoding** (a draft model proposes, the target verifies), **quantization** (FP8/INT8/AWQ/GPTQ weight compression), **tensor parallelism** (a layer's matrices sharded across GPUs), **LoRA** (low-rank adapters loadable per request), **guided/structured decoding** (constrain output to a grammar/JSON schema), and the **OpenAI-compatible server** (vllm serve exposing `/v1/chat/completions`). You do not need to understand their implementations — you need the retriever to find them; but recognizing the terms makes your debugging (“why did this question retrieve nothing sensible?”) dramatically faster.

What the corpus's shape implies for your system

- Docs are heading-rich MkDocs Markdown with fenced code examples — ideal for the heading-based chunker; the 80% docs threshold assumes you exploit that structure.
- Code files are large and class-heavy: AST chunking with per-method descent (Chapter 9) matters; naive fixed windows through `vllm/core/scheduler.py` will cost you code recall.
- Identifiers are long and compound (`max_num_batched_tokens`, `gpu_memory_utilization`): identifier splitting (Chapter 8) is not optional here.
- The repo contains non-text assets and huge generated files: filter by extension and file size at discovery, or burn your 5-minute indexing budget.

Note — Do not confuse the roles: vLLM is the *corpus*, not your runtime. You index vLLM's source; you run Qwen3-0.6B with HuggingFace transformers on CPU. (Amusingly, vLLM would be the right tool to serve Qwen at scale — a symmetry worth one smile at the defense, not an implementation choice.)

CHAPTER 14

One Question, End to End: A Worked Trace

Every concept from chapters 2-12, exercised once, on one realistic question, with real numbers at every step.

Theory consolidates when you push one concrete example through every stage by hand. This chapter traces the question “**How do I limit how much GPU memory vLLM uses?**” from raw string to graded answer. The numbers are simplified (a five-chunk universe instead of thirty thousand) but every formula, threshold, and data structure is exactly the real one.

Step 0 — what exists before the question arrives

Indexing already ran. Among the tens of thousands of chunks it produced, five matter for this trace:

id	file_path (under data/raw/vllm-0.10.1/)	span	flavour
C1	docs/configuration/optimization.md	(0, 1850)	prose: memory tuning guide, mentions <code>gpu_memory_utilization</code> twice
C2	vllm/config.py	(41200, 43050)	code: the config dataclass defining <code>gpu_memory_utilization</code> , docstring included
C3	vllm/engine/arg_utils.py	(9000, 10700)	code: CLI argument registration, one mention
C4	docs/serving/openai_compatible_server.md	(5400, 7300)	prose: server flags list, one mention
C5	benchmarks/README.md	(300, 2100)	prose: unrelated benchmark text, zero mentions, but says 'GPU' and 'memory' often

The analyzer (Chapter 8) processed each chunk at indexing time. For C2, the line “`gpu_memory_utilization: float = 0.9`” contributed the terms [`gpu_memory_utilization`, `gpu`, `memory`, `utilization`, `float`, `0`, `9`] — the intact identifier *and* its parts, lowercased. Document frequencies over the whole real corpus (illustrative but realistic orders of magnitude, $N = 30,000$ chunks): $df(gpu_memory_utilization)=40$, $df(gpu)=2,600$, $df(memory)=3,900$, $df(utilization)=210$, $df(limit)=1,100$, $df(vllm)=9,000$.

Step 1 — the query through the analyzer

The same function that indexed the corpus, applied to the query

```
raw:      "How do I limit how much GPU memory vLLM uses?"
tokens: [How, do, I, limit, how, much, GPU, memory, vLLM, uses]
split:   (no compounds present -> unchanged)
fold:    [how, do, i, limit, how, much, gpu, memory, vllm, uses]
stop:    [limit, gpu, memory, vllm, uses]      (short stoplist)
stem:    [limit, gpu, memori, vllm, use]       (if Snowball enabled)
```

Note what did *not* happen: the query never mentions `gpu_memory_utilization`. If the analyzer had indexed only intact identifiers, C2 would be unreachable from this query — vocabulary mismatch in its purest form. Because indexing emitted the parts too, the query terms `gpu`, `memory` (and `utilization`'s absence hurts nothing) connect to C2's postings.

Step 2 — BM25 scoring

The retriever walks the postings lists of the five query terms and accumulates scores. IDF's from the `df` values above ($N=30,000$, $\text{idf} = \ln((N-\text{df}+0.5)/(\text{df}+0.5)+1)$):

Corpus statistics feeding the formula ($k1=1.5$, $b=0.75$)

```
idf(limit) = ln((30000-1100+0.5)/(1100.5)+1) = ln(27.26) = 3.31
idf(gpu)   = ln((30000-2600+0.5)/(2600.5)+1) = ln(11.54) = 2.45
idf(memory)= ln((30000-3900+0.5)/(3900.5)+1) = ln( 7.69) = 2.04
idf(vllm)  = ln((30000-9000+0.5)/(9000.5)+1) = ln( 3.33) = 1.20
idf(uses)  ~ 2.0 (say df ~ 4000)
```

Term frequencies per chunk (after analysis):

	limit	gpu	memory	vllm	uses	
C1	2	4	6	3	1	d ~ avgdl
C2	1	5	5	2	0	d ~ 1.1 avgdl
C3	0	2	2	3	0	
C4	0	1	1	4	1	
C5	0	6	5	1	0	d ~ avgdl

Accumulated BM25 — ranking: $C1 > C2 > C5 > C4 > C3$

TF gain $g(\text{tf}) = \text{tf} * 2.5 / (\text{tf} + 1.5)$ at average length:

$g(1)=1.00$ $g(2)=1.43$ $g(3)=1.67$ $g(4)=1.82$ $g(5)=1.92$ $g(6)=2.00$

```
score(C1) = 3.31*1.43 + 2.45*1.82 + 2.04*2.00 + 1.20*1.67 + 2.0*1.0
           = 4.73 + 4.46 + 4.08 + 2.00 + 2.00 = 17.27
```

```
score(C2) = 3.31*1.00 + 2.45*1.92 + 2.04*1.92 + 1.20*1.43 (len adj.)
           ~ 3.31 + 4.70 + 3.92 + 1.72 = 13.65 -> ~13.1 after
           the 1.1x length normalization shaves the TF gains
```

```
score(C5) = 2.45*2.00 + 2.04*1.92 + 1.20*1.00 = 10.02
```

```
score(C3) = 2.45*1.43 + 2.04*1.43 + 1.20*1.67 = 8.42
```

```
score(C4) = 2.45*1.00 + 2.04*1.00 + 1.20*1.82 + 2.0*1.0 = 8.67
```

Read the ranking critically. C1 (the tuning guide) deservedly wins: it matches the rare, high-IDF term “limit” twice and everything else besides. C2 (the actual config definition) is second — correct and crucial. But look at C5: pure noise that outranks two genuinely relevant chunks just by repeating “GPU” and “memory”. That is lexical retrieval's honest face: with $k=5$ all five chunks ship anyway, recall survives, and the noise chunk becomes the generator's problem — which is why augmentation (Step 4) reorders and prunes rather than blindly trusting ranks 1..k.

Step 3 — the grader's view

Suppose the ground truth for this question lists two sources: $g1 = (\text{docs/configuration/optimization.md}, 400, 1600)$ and $g2 = (\text{vllm/config.py}, 41900, 42600)$. Your top-5, in order: C1, C2, C5, C4, C3. Matching per the moulinette (same file AND IoU ≥ 0.05):

IoU arithmetic exactly as Chapter 11 defined it

```
g1 vs C1: same file. inter = min(1850,1600)-max(0,400) = 1200
           union = 1850 + 1200 - 1200 = 1850   IoU = 0.649   FOUND
g2 vs C2: same file. inter = min(43050,42600)-max(41200,41900) = 700
           union = 1850 + 700 - 700 = 1850     IoU = 0.378   FOUND
```

```
recall@5(question) = 2/2 = 1.0
```

```
recall@1(question) = 1/2 = 0.5   (only g1 matched by top-1)
```

Notice `recall@1` is 0.5 *even though the top-1 result was perfect* — the question has two ground-truth sources and one result can only match one of them. Multi-source questions structurally cap recall at low k ; this is why dataset-level `recall@1` in the moulinette's illustrative output (0.450) looks so much worse than `recall@10` (0.720) even for a healthy system.

Step 4 — augmentation and the prompt

The answerer takes the top 5 retrieved sources but budgets ~8,000 context characters and feeds the best 3 (C1, C2, C4 — C5 dropped by a cheap heuristic or simply by budget order), each labelled with its path, best-first. The prompt follows Chapter 12's template; with thinking disabled, Qwen3-0.6B produces something like:

A grounded answer: every claim traceable to C1 or C2

Set the `gpu_memory_utilization` option (a float between 0 and 1, default 0.9) to cap the fraction of GPU memory vLLM pre-allocates. For example: `vllm serve MODEL --gpu-memory-utilization 0.7`, or `LLM(model=..., gpu_memory_utilization=0.7)` in Python. See `docs/configuration/optimization.md` and `vllm/config.py`.

Check it against the rubric: **coherent** (yes), **grounded** (the flag name, default, and range all appear in C1/C2; nothing invented), **on point** (answers 'how do I limit', not 'what is PagedAttention'). This, serialized into a `MinimalAnswer` with the five retrieved sources and wrapped in `StudentSearchResultsAndAnswer`, is a full-credit trajectory.

Step 5 — what failure would have looked like

- **Analyzer without identifier splitting:** C2's postings contain only `gpu_memory_utilization`; query terms `gpu/memory` never touch it; g2 missed; `recall@5` = 0.5 for this question. Multiply across a dataset and the 50% code bar fails. One regex was the difference.
- **Fixed 2000-char windows on config.py:** a window boundary at 42,000 splits the dataclass field from its docstring; both halves score weakly; maybe neither reaches top-5. AST chunking existed to prevent exactly this.
- **Paths stored as `vllm-0.10.1/docs/...`** (missing the `data/raw/` prefix): every single match fails the verbatim path comparison; `recall` = 0.000 everywhere; the moulinette output looks like your retriever is broken when actually one `os.path` call is.

- **Thinking mode left on:** the answer field opens with 400 tokens of deliberation; “coherent and on point” dies; time budget for answer_dataset triples.

Every one of these failure modes reappears, with detection procedures and fixes, in Volume 3. If the trace above felt comfortable, you have internalized Volume 1; if any step felt like magic, its chapter number is printed at the top of this one — go back before moving to Volume 2.

CHAPTER 15

Defense Drill: Forty Questions You Should Be Able to Answer

Peer-evaluation rewards people who can explain, not people who can paste. Drill these until they bore you.

Each question below is one an evaluator can reasonably ask after reading the subject. Model answers are deliberately compressed — two or three sentences, the density you should aim for out loud. Cross-references point to the chapter that treats the topic fully. Cover the answer column and rehearse.

Language models and RAG (Chapters 2-4)

Question	Compressed answer
What is a language model, in one sentence?	A probability distribution over the next token given the previous ones; generation is sampling from it in a loop.
Why does an LLM hallucinate?	Training maximizes likelihood of plausible text, not truth; where parametric memory is thin the model interpolates confidently, because refusing is out-of-distribution.
Why RAG instead of fine-tuning here?	Facts change faster and cheaper than weights: re-indexing is minutes on CPU, fine-tuning is GPU-hours, stays lossy, and risks forgetting; and reading is an easier task than remembering for a 0.6B model.
Why not paste the whole repo into the context?	Tens of MB vs a 32k-token window: it does not fit; and even within budget, attention degrades over long contexts (lost in the middle).
Name the four RAG stages and their artifacts.	Indexing (files->index), retrieval (question->top-k sources), augmentation (sources->prompt context), generation (context->grounded answer JSON).
What is the KV cache and why care?	Cached attention keys/values per token; memory grows linearly with context — the reason long contexts are costly, and the exact problem vLLM's PagedAttention manages.
What is a chat template?	The convention (ChatML for Qwen) that serializes system/user/assistant turns into one token sequence with special tokens; a system prompt is just early text in that sequence.

Retrieval theory (Chapters 5-7)

Question	Compressed answer
What is an inverted index?	A map term -> postings list (chunk ids + term counts); queries touch only their terms' lists instead of every document.

Question	Compressed answer
Define TF and IDF and why multiply them.	TF: repetition signals aboutness. IDF ($\log N/df$): rarity signals discriminative power. Their product weights matches by informativeness.
What two problems of TF-IDF does BM25 fix?	Linear TF growth (fixed by saturation via k_1) and unprincipled length handling (fixed by pivoted normalization via b).
What does k_1 do? What happens at $k_1=0$?	Controls TF saturation speed; the gain caps at k_1+1 . At $k_1=0$ term frequency is ignored entirely — pure presence/absence scoring.
What does b do? Why might you lower it here?	Interpolates length normalization from none (0) to proportional (1). Chunks are already capped near 2000 chars, so lengths are tight and strong normalization mostly adds noise.
Why does BM25's IDF have $+0.5$ and $+1$?	Half-counts smooth df near 0 and N ; the $+1$ keeps weights positive for very common terms, which would otherwise be penalized.
What is vocabulary mismatch and your mitigations?	Query and document name one concept differently. Mitigations: case folding, identifier splitting keeping both forms, optional stemming, and (bonus) dense/hybrid retrieval.
Why is a plain dict-based index fast enough?	~30k chunks, 5-term queries touch a few thousand postings; milliseconds per query against a 90s/200-question budget — no need for approximate structures.

Chunking, analysis, evaluation (Chapters 8-11)

Question	Compressed answer
Why two chunking strategies?	Meaning units differ: Markdown breaks at headings/paragraphs, Python at functions/classes (AST). One fixed window mis-cuts both.
Why must chunks stay ≤ 2000 characters?	It is the moulinette's <code>max_context_length</code> : one longer source invalidates the whole output file. Enforced by construction and asserted at serialization.
Exactly when does a retrieved source count?	Same <code>file_path</code> verbatim AND interval $IoU \geq 0.05$ against the ground-truth span, within the top k .
Two equal 2000-char spans: how much overlap for $IoU 0.05$?	About 190 characters — $inter/(4000-inter) \geq 0.05$ gives $inter \geq \sim 190.5$. Grazing contact does not count.
Why is recall (not precision) graded?	The retriever's job is to guarantee evidence presence; the generator tolerates extra passengers, but can never cite what was not retrieved.
What does a large $recall@10$ - $recall@5$ gap tell you?	The right chunks are found but ordered too low: fix scoring (k_1/b , boosts), not chunking.
Why can $recall@1$ be 0.5 with a perfect top hit?	A question with two ground-truth sources can match at most one with one result; the macro metric divides by $ G(q) $.
Why must offsets index the raw file text?	The grader computes IoU on the original file's character axis; offsets into any cleaned/normalized copy are shifted and silently fail.

Generation and system design (Chapters 12-13)

Question	Compressed answer
Why feed fewer chunks to the model than you retrieved?	The JSON reports retrieval; the prompt is editorial. Small models answer best from 3-5 clean snippets; the rest dilutes attention.
What is thinking mode and your policy?	Qwen3's optional <think> reasoning trace, toggled via the chat template. Disabled for dataset runs (speed, clean output); output stripped defensively regardless.
How do you fight hallucination in the answer stage?	Grounding instruction, an explicit 'say if absent' escape hatch, best-snippet-first ordering, short answers, low-ish temperature, and citations by file path.
What is PagedAttention (one line)?	vLLM's virtual-memory-style KV-cache management in fixed blocks, eliminating fragmentation to raise batch size and throughput.
Your corpus file selection and why?	.py via AST chunker, .md via heading chunker, small text/config files via text chunker; binaries and giant generated files skipped for the 5-minute budget. Inclusive by default because a coverage hole is unrecoverable recall.
Why Fire for the CLI?	Subject requirement; it maps a class's methods to subcommands and keyword arguments to flags with zero boilerplate — one object exposes index/search/answer/evaluate.

Note — Drill format that works: one peer reads questions in random order, the other answers aloud in under thirty seconds each. Anything that takes longer, re-read its chapter the same day. Repeat forty-eight hours before the defense — spaced repetition is not just for flashcards.

CHAPTER 16

The Subject, Decoded

Every requirement, mapped to the concept that satisfies it — a checklist view of everything this volume covered.

This closing chapter re-reads the subject as an engineer's requirements traceability matrix. For each requirement: where the theory lives in this volume, and where the implementation lives in Volumes 2–3.

Subject requirement	Theory (Vol 1)	Practice
Python 3.10+, flake8, mypy (given flags), docstrings, type hints	—	Vol 2 ch. 2, 14
uv as package manager; reviewer runs only uv sync	—	Vol 2 ch. 1
Makefile: install/run/debug/clean/lint(-strict)	—	Vol 2 ch. 1
pydantic data models (given schemas)	ch. 4 (data contracts)	Vol 2 ch. 3
CLI via Python Fire; tqdm progress bars	—	Vol 2 ch. 9-10
Two chunking strategies (Python / Markdown-text)	ch. 9	Vol 2 ch. 5-6
TF-IDF or BM25 (at least one)	ch. 6-7	Vol 2 ch. 7-8
max_chunk_size CLI arg, default 2000, hard cap	ch. 9, 11	Vol 2 ch. 5-6; Vol 3 ch. 3
Exact file_path, char ranges; k results	ch. 4, 11	Vol 2 ch. 8, 12
Answer generation with Qwen/Qwen3-0.6B	ch. 2, 12	Vol 2 ch. 11
Six CLI commands (index...evaluate)	ch. 4	Vol 2 ch. 9
Never crash on degenerate inputs	—	Vol 2 ch. 10; Vol 3 ch. 5
Recall@5: 80% docs / 50% code; IoU 0.05	ch. 11	Vol 3 ch. 1-3
Indexing <= 5 min; 200 questions <= 90 s	ch. 5, 10	Vol 3 ch. 4
Project layout for the automated run	—	Vol 2 ch. 1, 13
README with 7 extra sections	—	Vol 3 ch. 11
Five bonuses	ch. 10 (theory for 1-2)	Vol 3 ch. 6-10

The five decisions that determine your score

1. **What you index** (file selection): a hole in coverage is an unrecoverable recall loss. Err inclusive within the time budget.

2. **Where you cut** (chunking): structural boundaries + near-ceiling sizes + the 2000 cap enforced by construction.
3. **What a term is** (analyzer): lowercase, split identifiers, keep both forms, short stoplist, one function used by both indexer and retriever.
4. **How you score** (BM25, k1/b tuned per measurements, optional field boosts): defaults first, tune with your own evaluate against public datasets.
5. **How you prompt** (top 3-5 snippets, grounding instruction, escape hatch, thinking off, post-processing): quality of answers under a small model is a prompt-engineering result, and the rubric says so.

If you can narrate those five decisions — with the trade-offs from this volume and the measurements from Volume 3 — the theoretical part of your defense is already won. Volume 2 now builds the thing.

CHAPTER 17

Glossary and Further Reading

Glossary

Term	Definition
Analyzer	function mapping raw text to index terms (tokenize, split, fold, filter)
Augmentation	selecting/ordering/formatting retrieved chunks into the model context
Bag of words	text as a multiset of terms, order ignored
BM25	probabilistic lexical ranking: smoothed IDF x saturated TF x length normalization
BPE	byte-pair encoding; subword tokenization used by LLMs
Bi-encoder	embeds query and document independently; enables precomputation
Chunk	contiguous character span of one file; the retrievable unit
Context window	maximum tokens a model can attend over (32,768 for Qwen3-0.6B)
Cross-encoder	scores query+document jointly; accurate, slow; used for re-ranking
Dense retrieval	ranking by similarity of learned embedding vectors
df / IDF	document frequency; inverse document frequency $\log(N/df)$ (smoothed variants in practice)
Embedding	learned vector representation of text
Grounding	constraining an answer to provided evidence
Hallucination	fluent, confident, unsupported model output
Hybrid retrieval	fusing lexical and dense rankings (e.g. RRF)
IoU	intersection-over-union of character intervals; grader threshold 0.05
Inverted index	map term $\rightarrow$ postings list of (chunk, tf)
KV cache	cached attention keys/values; memory cost of long context
Lexical retrieval	ranking by term statistics (TF-IDF, BM25)
Moulinette	the provided grading executable
Postings list	the chunk ids (and counts) containing a term
Recall@k	share of ground-truth sources matched within top-k
RRF	reciprocal rank fusion: sum of $1/(K+\text{rank})$ across rankers
Stemming	reducing inflected forms to a stem (Porter/Snowball)
Stopwords	high-frequency low-information words dropped at analysis
TF saturation	diminishing returns of repeated terms (BM25's k1 factor)

Term	Definition
Thinking mode	Qwen3's optional chain-of-thought trace in <think> tags
Vocabulary mismatch	query and document express one concept with different words
Zipf's law	term frequency inversely proportional to frequency rank

Further reading

- Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond* (2009) — the definitive BM25 treatment.
- Manning, Raghavan & Schütze, *Introduction to Information Retrieval* (2008, free online) — chapters 1-8 cover everything in Chapters 5-7 here.
- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020) — the RAG paper.
- Karpukhin et al., *Dense Passage Retrieval* (2020) — dense retrieval for QA.
- Cormack, Clarke & Buettcher, *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods* (2009) — the RRF paper.
- Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention* (2023) — the vLLM paper; you are indexing its implementation.
- Qwen3 model card and technical report (Hugging Face / arXiv 2025) — sampling recommendations, thinking mode, context length.
- Liu et al., *Lost in the Middle* (2023) — positional attention decay; why snippet order matters.
- The reportlab of it all aside: the 42 subject itself, twice. Every constraint in this series traces to a sentence in it.

End of Volume 1. Volume 2 opens the editor: project bootstrap, data models, chunkers, index, CLI, generation — the complete build, in order, every line explained.

CHAPTER 18

Appendix: The Mathematical Toolbox

Every formula in this volume uses only four ideas. Here they are, self-contained, for readers who want the foundations airtight.

Logarithms, and why IR is full of them

$\log_b(x)$ answers “b to what power gives x?”. Natural log (ln, base e) is the IR convention; base only rescales by a constant, which never changes a ranking. Three properties do all the work in this volume: $\log(ab) = \log a + \log b$ (products become sums — a multi-term score can be read as a product of probabilities turned into a sum of weights); log grows without bound but ever more slowly (perfect for damping counts: the step from $df=1$ to $df=10$ matters far more than from $df=1001$ to $df=1010$); and $\log(1) = 0$ (a term in every document, $N/df = 1$, earns exactly zero weight — the math says “this term cannot distinguish anything”, which is the entire IDF idea in one identity).

In through the range of values this project produces

x:	1	2	5	10	100	1000	30000
ln(x):	0.00	0.69	1.61	2.30	4.61	6.91	10.31

Ratios of counts as probabilities

$df(t)/N$ estimates the probability that a random chunk contains t; the classic IDF is $-\log$ of it. Rewriting $idf = \log(N/df) = -\log(df/N)$ exposes IDF as **self-information** from information theory: the number of “surprise bits” (nats, for ln) in the event “this chunk contains t”. A term in 40 of 30,000 chunks carries $-\ln(0.00133) \sim 6.6$ nats; a term in 9,000 of 30,000 carries 1.2. BM25's version adds the 0.5 half-counts (a Bayesian correction for estimating probabilities from small counts) and the +1 (positivity), but the informational reading survives: **a match's weight is how surprising the match is**.

Saturating functions

The BM25 TF gain $tf \cdot (k1+1)/(tf+k1)$ is a scaled instance of $x/(x+c)$ — the simplest saturating curve: through the origin, slope steepest at zero, asymptote at 1 (here $k1+1$). The constant $c = k1$ is the half-saturation point: at $tf = k1$ the gain is exactly half its maximum. This one-parameter family (Michaelis–Menten kinetics in biochemistry, hyperbolic discounting in economics) is what you reach for whenever “more of x helps, with diminishing returns” needs a formula. Recognizing it lets you re-derive BM25's TF factor from scratch at a whiteboard instead of memorizing it.

$x/(x+c)$: half the asymptote is reached at $tf = k1$

tf:	0	1	2	3	5	10	inf
tf/(tf+1.5):	0	0.400	0.571	0.667	0.769	0.870	1.0
x2.5 (=k1+1 gain):	0	1.000	1.429	1.667	1.923	2.174	2.5

Vectors, dot products, cosine

A vector here is just an array of numbers — one per vocabulary term (sparse TF-IDF, dimension $\sim 10^5$, mostly zeros) or one per learned feature (dense embedding, dimension 384). The dot product $q \cdot d = \sum_i q_i d_i$ adds up coordinate-wise agreement; the norm $|q| = \sqrt{q \cdot q}$ is Euclidean length; and cosine similarity $q \cdot d / (|q||d|)$ is the dot product with length divided out — pure direction agreement, in $[-1, 1]$ (in $[0, 1]$ when all coordinates are non-negative, as in TF-IDF). Two texts about the same topic point the same way regardless of how long they are: that is the geometric sentence behind both Chapter 6's cosine-normalized TF-IDF and Chapter 10's embedding retrieval. Pre-normalize every vector to unit length and cosine becomes a bare dot product — one multiply-add per dimension, a matrix-vector product for a whole corpus, which is why brute-force dense search over 30,000 chunks is a millisecond-scale operation in numpy.

Cosine similarity by hand, once

```
q = (2, 0, 1), d = (4, 1, 2)
q.d = 2*4 + 0*1 + 1*2 = 10
|q| = sqrt(4+0+1) = 2.236 |d| = sqrt(16+1+4) = 4.583
cos = 10 / (2.236*4.583) = 0.976 (nearly same direction)
```

Intervals: intersection, union, IoU

For intervals $a = [a_0, a_1)$ and $b = [b_0, b_1)$: the intersection length is $\max(0, \min(a_1, b_1) - \max(a_0, b_0))$ — the min of the right ends minus the max of the left ends, floored at zero when they miss; the union length is $\text{len}(a) + \text{len}(b) - \text{inter}$ (inclusion–exclusion, counting the shared part once). IoU = inter/union is scale-free: it measures *proportional* agreement, so a 100-char match inside 100-char spans scores like a 1000-char match inside 1000-char spans. Chapter 11 built the grader's matching predicate on exactly these three lines; if you can reproduce them cold, you can re-implement your own evaluate command from memory.

The complete grading geometry, one worked line each

```
a = [0, 2000) b = [1500, 1900)
inter = max(0, min(2000, 1900) - max(0, 1500)) = 400
union = 2000 + 400 - 400 = 2000
IoU = 400/2000 = 0.20 >= 0.05 -> match
```

Averages that lie, averages that don't

The dataset score is a **macro** average: mean over questions of per-question recall. The alternative — micro averaging, pooling all ground-truth sources and dividing found by total — weights multi-source questions more heavily. The two can disagree noticeably when source counts vary; knowing which one the grader uses (macro) tells you where improvement pays: fixing one hopeless five-source question moves the macro mean exactly as much as fixing one easy one-source question. Optimization effort should therefore chase the *cheapest* failed questions first — a triage principle Volume 3 turns into a workflow.

Note — That is the entire mathematical surface of this project: logarithms, count ratios, one saturating curve, dot products, interval overlap, and a mean. Nothing here exceeds first-year material — the difficulty of RAG is engineering judgment, not mathematics. Volume 2 awaits.

CHAPTER 19

Appendix: Notation, Symbols, and a Study Plan

Symbols used throughout this volume

Symbol	Meaning	First appears
t	a term (normalized token)	ch. 5
d	a document -- in this project, a chunk	ch. 5
q	a query (bag of terms) or its vector	ch. 5
N	number of chunks in the corpus	ch. 6
df(t)	document frequency: chunks containing t	ch. 6
tf(t,d)	occurrences of t in chunk d	ch. 6
idf(t)	inverse document frequency weight of t	ch. 6
d	length of chunk d in tokens	ch. 7
avgdl	mean chunk length over the corpus	ch. 7
k1, b	BM25 saturation / length-normalization knobs	ch. 7
k	result-list cutoff (top-k)	ch. 11
G(q)	ground-truth sources of question q	ch. 11
IoU	intersection-over-union of character spans	ch. 11
E	matrix of chunk embeddings (num_chunks x dim)	ch. 10
K	the RRF damping constant (60)	ch. 10

A one-week study plan

For a pair working full-time on the project, this sequencing has worked well. Days 1-2: read this volume; run the moulinette on a hand-written dummy StudentSearchResults file to demystify the grader on day one. Days 2-4: build along Volume 2 in order -- bootstrap, models, chunkers, index, CLI, retrieval; get search_dataset producing valid JSON before touching generation. Day 5: generation and answer_dataset. Days 6-7: Volume 3 -- measure, tune chunking and BM25 against the public datasets, harden edge cases, write the README, drill the defense questions from chapter 15, then attempt bonuses strictly in the order semantic, hybrid, caching, incremental, API (value per effort, per Volume 3).

How the three volumes cross-reference

Citations of the form 'Vol 2 ch. 7' point at the implementation of an idea introduced here; 'Vol 3 ch. 3' points at its measurement and tuning. The dependency is intentional and one-directional: nothing in this volume requires code, everything in Volume 2 assumes the theory here, and everything in Volume 3 assumes a working Volume 2 build. If a Volume 2 listing ever feels arbitrary, its justification lives in a chapter of this book; the chapter-16 traceability table maps every subject requirement to both.

Note — Errata policy for your own notes: when a measured number in Volume 3 contradicts a heuristic stated here (it can -- corpora have personalities), trust the measurement, and write down why. That habit -- theory as prior, measurement as posterior -- is the single most transferable skill this project teaches.